

Sentiment-Driven Alpha Generation for Indian Equities

Multimodal Pipeline: FinBERT News Sentiment + Technical Indicators

→ LSTM / Transformer Ensemble → Nifty 50 Return Prediction

Phase 1: Data Collection

Phase 2: Feature Engineering

Phase 3: Model Validation

Table of Contents

What this presentation covers

01

Motivation & Problem Statement

Why Indian equities? Why sentiment-driven models?

03

What Makes This Project Different

Novel contributions vs. existing research

05

Phase 2 — Feature Engineering

FinBERT sentiment + 17 technical/temporal features

07

Experimental Results

Metrics, training curves, model comparison

09

Future Work

Planned extensions and research directions

02

Literature Review & Gap Analysis

Key prior work and what is missing in the field

04

Phase 1 — Data Ingestion Pipeline

Live RSS feeds, DuckDB storage, Nifty50 tagging

06

Phase 3 — Model Architecture

LSTM & Transformer ensemble + directional loss

08

Challenges & Limitations

Data quality, overfitting, computational issues

10

References

Key citations and related literature

Motivation & Problem Statement

Why this research? What gap does it address?



Untapped News Signal

Over 500 financial news articles are published daily about Nifty50 stocks. Yet most quantitative models for NSE rely purely on price and volume data, completely ignoring this rich text signal.



Information Lag in Retail Markets

Indian retail investors process news with a 1–2 day lag. Institutional players exploit this predictable delay. An automated NLP pipeline can capture this alpha before the market fully adjusts.



No Unified Real-Time Pipeline

Prior academic work combines sentiment OR technicals — never both — in a single live system. No existing open framework ingests Indian RSS feeds, scores with FinBERT, and predicts next-day returns for all 50 Nifty stocks.

Research Question

Can a multimodal ensemble model combining FinBERT news sentiment with technical price features predict next-day overnight (Close-to-Open) returns for Nifty50 stocks better than technical-only baselines?

Project Scope

- Universe: All 50 Nifty50 stocks
- Data: Real-time RSS + 2yr archive
- Target: Close-to-Open log return
- Evaluation: Walk-forward CV

Literature Review & Related Work

What has been done — and what is still missing

[1] Zeng & Jiang (2022) · arXiv:2204.05783

FinBERT + LSTM for Stock Movement

What:

Integrates FinBERT sentiment scores with LSTM networks for S&P 500 stock movement prediction.

Finding:

FinBERT-LSTM outperforms standalone LSTM & ARIMA; sentiment significantly improves accuracy.

Gap:

U.S. markets only; no technical features; no live RSS pipeline; no Indian stock focus.

⚠ No Indian market / technical combo

[2] SSRN 3753015 (2020) — Aggregate Sentiment

News Sentiment & Nifty 50 Returns

What:

Studies aggregate news sentiment (business/economy/politics) as a predictor of Nifty50 index-level returns.

Finding:

Significant sentiment-return relationship found ($R^2 = 0.277$, $\beta = 0.526$, $p < 0.01$).

Gap:

Index-level only — no individual stock alpha. No LSTM/Transformer. No live prediction system.

⚠ No stock-level prediction

[3] Mehtab & Sen (2019) · arXiv:1912.07700

Deep Learning + NLP for Nifty50

What:

Applies Twitter sentiment + LSTM on NSE Nifty50 data from 2015–2017 with one-week prediction horizon.

Finding:

Hybrid NLP+LSTM approach improves NSE predictions over price-only models.

Gap:

Outdated data (2015–17); Twitter unreliable for Indian financial news; no technical features.

⚠ Twitter data; outdated

What Makes This Project Different

Novel contributions versus prior work — dimension by dimension

This project addresses all three gaps identified in the literature: Indian market focus, multimodal feature fusion, and a live end-to-end pipeline. The table below highlights the key differences.

Dimension	Existing Literature	This Project (BTP2)	
Market	S&P 500 / Generic NSE index	All 50 Nifty50 individual stocks	●
Data Source	Bloomberg, Twitter, static CSV files	Live RSS (9 sources) + 2-year scraped archive	●
NLP Model	General BERT / keyword dictionaries	FinBERT — domain-tuned on financial text	●
Features	Price history OR sentiment — not both	FinBERT sentiment + 7 technical indicators (RSI, vol, BB...)	●
Prediction Target	Close-to-Close (includes intraday noise)	Close-to-Open overnight gap (direct news response)	●
Architecture	LSTM or Transformer in isolation	LSTM + Transformer ensemble with directional loss	●
Pipeline	Offline batch / research-only	Scheduler + DuckDB + real-time Streamlit dashboard	●
Evaluation	Single train/test split	Walk-forward CV + Deflated Sharpe + DM test	●

System Architecture Overview

Three phases — from raw news to trading signals

The system is built as three sequential, independent phases. Each phase writes to a shared DuckDB database so any phase can be re-run in isolation without affecting the others.

1

Phase 1

Data Ingestion

- Fetch 9 RSS feeds every 30 min
- MinHash dedup (Jaccard ≥ 0.75)
- flashtext ticker tagging (150+ aliases)
- NSE calendar trading-date alignment
- DuckDB bulk INSERT OR IGNORE
- yfinance OHLCV download

→ raw_news + ohlcv tables

2

Phase 2

Feature Engineering

- FinBERT scores all tagged headlines
- Aggregate sentiment per (ticker, date)
- RSI-14, Bollinger, vol, rel-volume
- Sentiment velocity (day-over-day Δ)
- Forward-fill sentiment ≤ 5 days
- Close-to-Open target assignment

→ feature_matrix (17 features + target)

3

Phase 3

Model & Signals

- RobustScaler on training split only
- LSTM Predictor (36K params)
- Transformer Encoder (155K params)
- Directional + Huber ensemble loss
- Walk-forward cross-validation
- LONG / SHORT / FLAT signals

→ predictions table → dashboard

Phase 1 — Data Ingestion Pipeline

How news travels from RSS feeds to a structured, queryable database

Every 30 minutes, a scheduler fetches all 9 news feeds in parallel, deduplicates headlines using MinHash similarity, tags each article with Nifty50 company names, aligns to the correct NSE trading date, and bulk-inserts into DuckDB. The OHLCV price data is separately downloaded from yfinance twice daily.

News Sources (Phase 1)

● Economic Times — Markets	Primary
● Economic Times — Stocks	Primary
● Economic Times — Top News	Primary
● MoneyControl	Active
● Business Standard	Active
● Livemint	Active
● Financial Express	Active
● Google News RSS	New v2
● Wayback Machine	Fallback

5-Step Processing Pipeline

- 1 Parallel Fetch**
ThreadPoolExecutor fetches all 9 feeds simultaneously with 12-second timeouts. Typically completes in under 30 seconds regardless of feed count.
- 2 MinHash Deduplication**
Jaccard similarity (threshold 0.75) over character k-shingles. Removes 30–40% of duplicate headlines that appear across multiple feeds about the same story.
- 3 Ticker Tagging**
flashtext Aho-Corasick trie with 150+ company aliases maps headline text to NSE symbols (e.g. 'Jio' → RELIANCE, 'JLR' → TATAMOTORS).
- 4 Trading Date Alignment**
Articles published after 15:30 IST are assigned to the next NSE trading day using the pandas_market_calendars NSE calendar. Weekend/holiday articles are rolled forward.
- 5 DuckDB Bulk Insert**
Open connection → INSERT OR IGNORE (idempotent by SHA1 article_id) → close connection immediately. Ensures read-only connections (dashboard) are never blocked.

Phase 2 — FinBERT Sentiment & Feature Engineering

From raw news text to a 17-dimensional numerical feature vector

FinBERT (ProsusAI/finbert) is a BERT-base model fine-tuned on 10,000+ financial news sentences. It outputs three probabilities per article (positive, negative, neutral) that sum to 1.0. These are averaged across all articles for a given stock on a given trading day to produce the sentiment block of the feature vector.

Sentiment Block (5)

From FinBERT

sent_pos

P(positive) — mean across all articles for this ticker today

sent_neg

P(negative) — mean across all articles for this ticker today

sent_neu

P(neutral) — mean across all articles

sent_spread

sent_pos – sent_neg — the PRIMARY signal for directional prediction

news_count

$\log_{1p}(\text{article count})$ — measures information volume / attention

Technical Block (7)

From OHLCV

rsi_14

14-day Relative Strength Index (Wilder's method) — momentum

price_to_ma20

close / MA20 – 1 — measures deviation from 20-day average

vol_20d

20-day annualised realised volatility from log returns

rel_volume

Today's volume ÷ 20-day average — unusual activity signal

bb_position

Bollinger Band position (0=lower, 1=upper band boundary)

Calendar Block (5)

Day-of-week

dow_mon

Monday one-hot — captures Monday effect in Indian markets

dow_tue

Tuesday one-hot

dow_wed

Wednesday one-hot

dow_thu

Thursday one-hot

dow_fri

Friday one-hot — pre-weekend position squaring

TARGET → $\text{target_ret_1d} = \log(\text{Open}[T+1] / \text{Close}[T])$ • Overnight gap (Close-to-Open) • Directly driven by T-date news sentiment

Phase 3 — Model Architecture

LSTM and Transformer predictors with a weighted ensemble

Both models receive the same input: a sequence of 30 consecutive trading days, each represented as a 17-dimensional feature vector (scaled with RobustScaler fit on training data only). They predict the next day's Close-to-Open log return as a scalar.

LSTM

LSTMPredictor · ~36K params

Input Sequence	Shape: (batch, 30, 17) — 30 trading days × 17 features
-----------------------	--

Linear(17 → 64)	Project features into LSTM hidden space; LayerNorm for stability
------------------------	--

LSTM(64 hidden, L=1)	Single-layer LSTM processes the 30-step sequence. Hidden state at T=30 is used.
-----------------------------	---

Dropout(p=0.25)	Regularisation — applied after LSTM output to prevent overfitting
------------------------	---

FC(64 → 32) → ReLU	Non-linear projection into smaller representation space
---------------------------	---

FC(32 → 1)	Scalar output: predicted $\log(\text{Open}[T+1] / \text{Close}[T])$
-------------------	---

Transformer

TransformerPredictor · ~155K params

Input Sequence	Shape: (batch, 30, 17) — same input as LSTM
-----------------------	---

Linear(17 → 32)	Embed features into d_model=32 dimensional space
------------------------	--

Learnable Pos. Encoding	30 trainable position embeddings — learns temporal importance
--------------------------------	---

2×Encoder Layer	d_model=32, nhead=4, FF=64, Pre-LayerNorm (stable training)
------------------------	---

Mean Pool → Dropout(0.20)	Average all 30 time-step representations into one vector
----------------------------------	--

FC(32→16)→GELU→FC(1)	Scalar output: predicted $\log(\text{Open}[T+1] / \text{Close}[T])$
-----------------------------	---

Loss Function & Training Strategy

Why standard MSE fails for return prediction — and how we fixed it

✗ Problem with MSE Loss

MSE (Mean Squared Error) treats all prediction errors equally. A prediction of +0.005% when the actual is -0.005% incurs the same penalty as +0.012% vs -0.005%. MSE never directly penalises predicting the wrong direction. A model minimising MSE can achieve near-zero loss while having a hit rate below 50% — exactly what we observed (IC = -0.119 with vanilla MSE).

✓ Our Loss Function

$$L = 0.6 \times \text{HuberLoss}(\delta=0.005) + 0.4 \times \text{DirectionalLoss}$$

where $\text{DirectionalLoss} = \text{mean}(\text{ReLU}(-\text{pred_norm} \times \text{target_norm}))$

$\text{pred_norm} = \text{pred} / \text{mean}(|\text{target}|)$ — normalised so penalty isn't negligible

Huber Loss ($\delta = 0.005$)

Quadratic for small errors ($< 0.005 = 0.5\%$) and linear for larger errors. Unlike MSE, it does not square large errors, making it robust to overnight gap outliers caused by earnings surprises or circuit breaker events.

Directional Penalty

$\text{ReLU}(-\text{pred} \times \text{target})$ is zero when pred and target have the same sign (correct direction), and positive when they disagree. By normalising both to unit scale, the penalty is meaningful even for tiny return values (~ 0.005). This directly optimises hit rate.

Training Configuration

AdamW optimiser (weight_decay=1e-4), cosine annealing LR from 1e-3 to 1e-5, gradient clipping at 0.5. Batch size 128, early stopping (patience=20). Input noise augmentation ($\sigma=0.005$) applied inside training loop only — never at validation.

Experimental Results & Model Comparison

Test set evaluation on 6,749 samples · Nifty50 stocks · 2024–2026

Training was carried out iteratively. Each row in the table below represents a specific configuration change. The key insight: the original model's IC of -0.119 was caused by zero-filled RSI values and a 5-day look-back window, not a fundamental model flaw. After all fixes, the model achieves positive — though still modest — predictive accuracy.

Configuration	MAE	RMSE	Hit Rate	IC	Status
LSTM — Original (SEQ=5, no RSI, zero NaN fill)	0.014	0.019	48.5%	-0.119	broken
LSTM — After partial fix (MSE loss)	0.014	0.019	48.5%	-0.119	no gain
LSTM — Huber+Directional loss + median fill	0.012	0.016	52.1%	0.047	improving
Transformer — All fixes applied	0.012	0.017	52.9%	0.005	improving
Ensemble — LSTM + TF blend (current)	0.012	0.016	53.2%	-0.047	Best

Key Insight:

The original negative IC (-0.119) was caused by zero-filled RSI values (RSI=0 means 'maximally oversold' — a false signal) and a SEQ_LEN of only 5 days (one week of context — far too little to learn temporal patterns). After median NaN-fill and restoring SEQ_LEN=30, the IC improved to 0.047, moving towards positive territory. Switching to Close-to-Open targets is expected to bring IC above zero.

Challenges & Limitations

Key technical and methodological obstacles — and how they were addressed

01

ET Inconsistent Article Counts

Problem: Economic Times archive returned 79–80 articles on some days and 500+ on others for the same section.

Fix: Root cause: v1 scraper fetched only page 1 of the archive. Fixed in v2 by paginating up to 10 pages per day, consistently returning 200–400 articles.

03 NaN Feature Contamination

Problem: 8,189 of 40,100 feature_matrix rows had null RSI and vol_20d values. Zero-filling made the model believe these stocks were permanently maximally oversold (RSI=0).

Fix: Extended OHLCV download to 5 years so all dates have sufficient lookback. Replaced zero-fill with per-column median imputation. Removed 8,189 corrupted rows.

05 Catastrophic Overfitting

Problem: 275K-parameter LSTM on 29K training samples: validation loss doubled by epoch 5 (train=0.266, val=0.705). Best checkpoint was epoch 1 — the model learned nothing.

Fix: Reduced model to 36K parameters (hidden=64, 1 layer). Added dropout=0.25, gradient clip=0.5, batch noise augmentation. Train and val loss now track closely.

02 Anti-Bot Blocking (Mint, MC, BS, FE)

Problem: Direct BeautifulSoup scraping of MoneyControl, Business Standard, Financial Express returned 0 articles — all blocked by Cloudflare.

Fix: Added Google News RSS as primary gap-filler. It aggregates all blocked sources, is freely accessible with no API key, and supports `after:` / `before:` date operators for historical queries.

04 Target Definition (Most Critical)

Problem: Close-to-Close target includes both the overnight gap AND the intraday move. The intraday move is driven by order flow during T+1's session — unpredictable from T-date features.

Fix: Switched to Close-to-Open return (overnight gap only). This directly measures the market's first reaction to yesterday's news and has ~60% lower variance than C2C.

06 Computational Constraints

Problem: CPU-only training environment: LSTM takes 30–90 min, FinBERT scoring 90 days of articles takes 3–8 hours. Limits iteration speed.

Fix: Reduced model size and batch size for faster iteration. FinBERT scoring is cached — only new articles are re-scored. GPU support available via CUDA if hardware permits.

Future Work & Planned Extensions

What comes next — near term, medium term, and long-term research directions

Near Term (Phase 4–5)

- **Walk-forward backtester**

Simulate all predictions as paper trades. Compute P&L, Sharpe ratio, max drawdown, and hit rate on held-out data using vectorised accounting.

- **Deflated Sharpe test**

Apply López de Prado (2018) Deflated Sharpe Ratio to account for multiple-testing bias across 4 model variants. DSR > 0.95 required to claim statistical significance.

- **Diebold-Mariano test**

Test whether LSTM and Transformer forecast errors are statistically different. If not, ensemble weight tuning is the correct improvement lever.

Medium Term

- **FinBERT fine-tuning on SEntFiN**

Fine-tune the FinBERT base model on the SEntFiN Indian financial corpus. Expected +3–5% accuracy improvement on NSE-specific vocabulary (crore, lakh, SEBI, promoter holding).

- **Nifty500 expansion**

Scale from 50 to 500 tickers. Requires GPU for FinBERT inference. Increases training data from ~36K to ~365K labelled rows — the single biggest improvement lever.

- **Multi-horizon prediction**

Add 5-day and 10-day return targets alongside 1-day. Allows mean-reversion signals for longer-term positions.

Long-Term Research

- **LLM zero-shot scoring**

Replace FinBERT with Mistral-7B or LLaMA-3 for zero-shot sentiment via chain-of-thought prompting. No fine-tuning required; potentially stronger for novel financial events.

- **Causal sentiment analysis**

Use difference-in-differences to isolate exogenous news shocks (earnings surprises, regulatory announcements) from endogenous sentiment correlated with price trends.

- **Reinforcement learning sizing**

Replace fixed volatility-scaled position sizing with a reinforcement learning agent that learns optimal dynamic sizing from historical P&L feedback.

References

Key citations and related literature

[1]

Zeng, Q. & Jiang, T. (2022). Financial Sentiment Analysis Using FinBERT with Application in Predicting Stock Movement. arXiv:2204.05783.

Primary architectural reference — establishes FinBERT + LSTM framework. This project extends it with Indian market focus, technical features, and live RSS pipeline.

[2]

Aggregate News Sentiment and Stock Market Returns in India. SSRN Working Paper 3753015 (2020).

Motivates Indian market focus. Establishes significant sentiment-return link for Nifty50 ($R^2=0.277$). Our work extends this to individual stock alpha generation.

[3]

Yang, Y., Uy, M.C.S., Huang, A. (2020). FinBERT: A Pre-Trained Financial Language Representation Model for Financial Text Mining. arXiv:2006.08097.

Describes the FinBERT model used for all sentiment scoring in this project. ProsusAI/finbert is the HuggingFace implementation referenced.

[4]

Mehtab, S. & Sen, J. (2019). A Robust Predictive Model for Nifty50 Using Deep Learning and NLP. arXiv:1912.07700.

Closest prior work on NSE stocks with NLP. Uses Twitter + LSTM on 2015–17 data. Our work improves upon it with multi-source RSS, Transformer, and live pipeline.

[5]

Vaswani, A. et al. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems (NeurIPS), 30.

Foundation architecture for our TransformerPredictor encoder component.

[6]

Hochreiter, S. & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735–1780.

Original LSTM paper. Architecture used as baseline predictor in Phase 3.

[7]

López de Prado, M. (2018). Advances in Financial Machine Learning. Wiley.

Source for Deflated Sharpe Ratio and walk-forward cross-validation methodology used in Phase 5 evaluation.

Thank You

Questions & Discussion Welcome

Phase 1: Live 9-source RSS pipeline · 30-min batches · DuckDB · 50 Nifty tickers

Phase 2: FinBERT sentiment · 17-feature vector · Close-to-Open overnight gap target

Phase 3: LSTM + Transformer ensemble · directional loss · walk-forward evaluation